Day 17 — Window Functions : Scenario: Find top 3 students per course and latest policy per customer

COMMANDS :

```
teja@teja72:~/day16-hospital-revenue$ cd ~
mkdir -p ~/day17-window-functions/src/main/scala
cd ~/day17-window-functions
teja@teja72:~/day17-window-functions$ nano build.sbt
teja@teja72:~/day17-window-functions$ nano src/main/scala/WindowFunctions.scala
teja@teja72:~/day17-window-functions$ export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export SPARK_LOCAL_IP=127.0.0.1
teja@teja72:~/day17-window-functions$ cd ~/day17-window-functions
teja@teja72:~/day17-window-functions$ sbt -java-home /usr/lib/jvm/java-17-openjdk-amd64 compile
[info] entering thin client - BEEP WHIRR
[info] starting sbt server in the background
[info] use 'sbt shutdown' to shutdown the server
[info]
[info] Updated file /home/teja/day17-window-functions/project/build.properties: set sbt.version to 2.0.8
[info] welcome to sbt 2.0.8 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /home/teja/day17-window-functions/project
[info] set current project to Day17-Window-Functions (in build file:/home/teja/day17-window-functions/)
[info] compiling 1 Scala source to /home/teja/day17-window-functions/target/out/jvm/scala-2.12.18/day17-window-functions/classes ...
[success] elapsed time: 9 s
teja@teja72:~/day17-window-functions$ sbt -java-home /usr/lib/jvm/java-17-openjdk-amd64 run
```

CONTENT IN NANO SBT FILE :

```
ThisBuild / scalaVersion := "2.12.18"

lazy val root = (project in file("."))
  .settings(
    name := "Day17-Window-Functions",
    version := "1.0",

    libraryDependencies ++= Seq(
      "org.apache.spark" %% "spark-core" % "3.5.3",
      "org.apache.spark" %% "spark-sql" % "3.5.3"
    ),

    Compile / run / fork := true,

    Compile / run / javaOptions ++= Seq(
      "--add-exports=java.base/sun.nio.ch=ALL-UNNAMED",
      "--add-opens=java.base/sun.nio.ch=ALL-UNNAMED"
    )
  )
```

SCALA PROGRAMM :

```
import org.apache.spark.sql.SparkSession
```

```
import org.apache.spark.sql.expressions.Window
import org.apache.spark.sql.functions._

object WindowFunctions {

  def main(args: Array[String]): Unit = {

    // ========================================================
    // CREATE SPARK SESSION
    // ========================================================

    val spark = SparkSession.builder()
      .appName("Day17 Window Functions")
      .master("local[*]")
      .getOrCreate()

    spark.sparkContext.setLogLevel("WARN")

    import spark.implicits._

    // ========================================================
    // STUDENT DATA
    // ========================================================

    val studentData = Seq(
      ("S001", "Teja",   "BigData",  92),
      ("S002", "Rahul",  "BigData",  88),
      ("S003", "Priya",  "BigData",  95),
      ("S004", "Anil",   "BigData",  88),
      ("S005", "Kiran",  "BigData",  82),

      ("S006", "Sneha",  "Cloud",    91),
      ("S007", "Arjun",  "Cloud",    95),
      ("S008", "Meena",  "Cloud",    89),
      ("S009", "Ravi",   "Cloud",    95),
      ("S010", "Pooja",  "Cloud",    84),

      ("S011", "Vijay",  "AI",       90),
      ("S012", "Swathi", "AI",       96),
      ("S013", "Nikhil", "AI",       90),
      ("S014", "Divya",  "AI",       94),
      ("S015", "Manoj",  "AI",       85)
    )
```

```
val students = studentData.toDF(
  "student_id",
  "student_name",
  "course",
  "score"
)

println("\n========== STUDENT DATA ==========")

students.show(false)

// ============================================================
// WINDOW DEFINITION
//
// Partition students by course.
// Within each course sort by score descending.
// ============================================================

val courseWindow =
  Window
    .partitionBy("course")
    .orderBy(desc("score"))

// ============================================================
// 1. ROW_NUMBER
//
// Gives unique sequential number.
// ============================================================

println("\n========== ROW_NUMBER ==========")

val rowNumberResult = students
  .withColumn(
    "row_number",
    row_number().over(courseWindow)
  )
  .orderBy(
    col("course"),
    col("row_number")
  )

rowNumberResult.show(false)

// ============================================================
```

```
// 2. RANK
//
// Equal scores get same rank.
// Gaps can occur after ties.
// ============================================================

println("\n========== RANK ==========")

val rankResult = students
  .withColumn(
    "rank",
    rank().over(courseWindow)
  )
  .orderBy(
    col("course"),
    col("rank")
  )

rankResult.show(false)

// ============================================================
// 3. DENSE_RANK
//
// Equal scores get same rank.
// No gaps after ties.
// ============================================================

println("\n========== DENSE_RANK ==========")

val denseRankResult = students
  .withColumn(
    "dense_rank",
    dense_rank().over(courseWindow)
  )
  .orderBy(
    col("course"),
    col("dense_rank")
  )

denseRankResult.show(false)

// ============================================================
// 4. TOP 3 STUDENTS PER COURSE
//
```

```
// Use row_number and filter rank <= 3.
// ============================================================

println("\n========== TOP 3 STUDENTS PER COURSE ==========")

val top3Students = students
  .withColumn(
    "student_rank",
    row_number().over(courseWindow)
  )
  .filter(col("student_rank") <= 3)
  .orderBy(
    col("course"),
    col("student_rank")
  )

top3Students.show(false)

// ============================================================
// 5. LAG
//
// Get the previous student's score
// within the same course.
// ============================================================

println("\n========== LAG - PREVIOUS SCORE ==========")

val lagResult = students
  .withColumn(
    "previous_score",
    lag("score", 1).over(courseWindow)
  )
  .orderBy(
    col("course"),
    desc("score")
  )

lagResult.show(false)

// ============================================================
// 6. LEAD
//
// Get the next student's score
// within the same course.
```

```
// ====================================================

println("\n========== LEAD - NEXT SCORE ==========")

val leadResult = students
  .withColumn(
    "next_score",
    lead("score", 1).over(courseWindow)
  )
  .orderBy(
    col("course"),
    desc("score")
  )

leadResult.show(false)

// ====================================================
// 7. LAG + LEAD TOGETHER
// ====================================================

println("\n========== PREVIOUS AND NEXT SCORE ==========")

val comparisonResult = students
  .withColumn(
    "previous_score",
    lag("score", 1).over(courseWindow)
  )
  .withColumn(
    "next_score",
    lead("score", 1).over(courseWindow)
  )
  .orderBy(
    col("course"),
    desc("score")
  )

comparisonResult.show(false)

// ====================================================
// CUSTOMER POLICY DATA
// ====================================================

val policyData = Seq(
  ("C001", "Teja",  "Health Basic",    "2025-01-10"),
```

```
  ("C001", "Teja",  "Health Premium",  "2025-06-15"),
  ("C001", "Teja",  "Health Gold",     "2026-02-20"),

  ("C002", "Rahul", "Life Basic",      "2025-03-12"),
  ("C002", "Rahul", "Life Premium",    "2026-01-25"),

  ("C003", "Priya", "Auto Basic",      "2025-02-18"),
  ("C003", "Priya", "Auto Premium",    "2025-09-10"),
  ("C003", "Priya", "Auto Gold",       "2026-03-05"),

  ("C004", "Anil",  "Home Basic",      "2025-05-20"),
  ("C004", "Anil",  "Home Premium",    "2026-04-01"),

  ("C005", "Kiran", "Travel Basic",    "2026-02-15")
)

val policies = policyData.toDF(
  "customer_id",
  "customer_name",
  "policy_name",
  "policy_date"
)

println("\n========== CUSTOMER POLICY DATA ==========")

policies.show(false)

// ========================================================
// CONVERT POLICY DATE FROM STRING TO DATE
// ========================================================

val policiesWithDate = policies
  .withColumn(
    "policy_date",
    to_date(col("policy_date"))
  )

// ========================================================
// POLICY WINDOW
//
// Partition by customer.
// Latest policy comes first.
// ========================================================
```

```
val policyWindow =
  Window
    .partitionBy("customer_id")
    .orderBy(desc("policy_date"))

// ========================================================
// 8. LATEST POLICY PER CUSTOMER
// ========================================================

println("\n========== LATEST POLICY PER CUSTOMER ==========")

val latestPolicy = policiesWithDate
  .withColumn(
    "policy_rank",
    row_number().over(policyWindow)
  )
  .filter(col("policy_rank") === 1)
  .orderBy("customer_id")

latestPolicy.show(false)

// ========================================================
// 9. CUSTOMER POLICY HISTORY
//
// Show previous policy for each customer.
// ========================================================

println("\n========== CUSTOMER POLICY HISTORY ==========")

val policyHistoryWindow =
  Window
    .partitionBy("customer_id")
    .orderBy("policy_date")

val policyHistory = policiesWithDate
  .withColumn(
    "previous_policy",
    lag("policy_name", 1).over(policyHistoryWindow)
  )
  .withColumn(
    "next_policy",
    lead("policy_name", 1).over(policyHistoryWindow)
  )
```

```
      .orderBy(
        col("customer_id"),
        col("policy_date")
      )

    policyHistory.show(false)

    // ============================================================
    // FINAL OUTPUT
    // ============================================================

    println("\n================================================")
    println("DAY 17 COMPLETED SUCCESSFULLY")
    println("================================================")

    spark.stop()
  }
}
```

OUTPUTS :

```
========== STUDENT DATA ==========
+----------+------------+-------+-----+
|student_id|student_name|course |score|
+----------+------------+-------+-----+
|S001      |Teja        |BigData|92   |
|S002      |Rahul       |BigData|88   |
|S003      |Priya       |BigData|95   |
|S004      |Anil        |BigData|88   |
|S005      |Kiran       |BigData|82   |
|S006      |Sneha       |Cloud  |91   |
|S007      |Arjun       |Cloud  |95   |
|S008      |Meena       |Cloud  |89   |
|S009      |Ravi        |Cloud  |95   |
|S010      |Pooja       |Cloud  |84   |
|S011      |Vijay       |AI     |90   |
|S012      |Swathi      |AI     |96   |
|S013      |Nikhil      |AI     |90   |
|S014      |Divya       |AI     |94   |
|S015      |Manoj       |AI     |85   |
+----------+------------+-------+-----+
```

```
========== ROW_NUMBER ==========
+----------+------------+-------+-----+----------+
|student_id|student_name|course |score|row_number|
+----------+------------+-------+-----+----------+
|S012      |Swathi      |AI     |96   |1         |
|S014      |Divya       |AI     |94   |2         |
|S011      |Vijay       |AI     |90   |3         |
|S013      |Nikhil      |AI     |90   |4         |
|S015      |Manoj       |AI     |85   |5         |
|S003      |Priya       |BigData|95   |1         |
|S001      |Teja        |BigData|92   |2         |
|S002      |Rahul       |BigData|88   |3         |
|S004      |Anil        |BigData|88   |4         |
|S005      |Kiran       |BigData|82   |5         |
|S007      |Arjun       |Cloud  |95   |1         |
|S009      |Ravi        |Cloud  |95   |2         |
|S006      |Sneha       |Cloud  |91   |3         |
|S008      |Meena       |Cloud  |89   |4         |
|S010      |Pooja       |Cloud  |84   |5         |
+----------+------------+-------+-----+----------+
```

```
========== RANK ==========
+----------+------------+-------+-----+----+
|student_id|student_name|course |score|rank|
+----------+------------+-------+-----+----+
|S012      |Swathi      |AI     |96   |1   |
|S014      |Divya       |AI     |94   |2   |
|S011      |Vijay       |AI     |90   |3   |
|S013      |Nikhil      |AI     |90   |3   |
|S015      |Manoj       |AI     |85   |5   |
|S003      |Priya       |BigData|95   |1   |
|S001      |Teja        |BigData|92   |2   |
|S002      |Rahul       |BigData|88   |3   |
|S004      |Anil        |BigData|88   |3   |
|S005      |Kiran       |BigData|82   |5   |
|S007      |Arjun       |Cloud  |95   |1   |
|S009      |Ravi        |Cloud  |95   |1   |
|S006      |Sneha       |Cloud  |91   |3   |
|S008      |Meena       |Cloud  |89   |4   |
|S010      |Pooja       |Cloud  |84   |5   |
+----------+------------+-------+-----+----+
```

```
========== DENSE_RANK ==========
+----------+------------+-------+-----+----------+
|student_id|student_name|course |score|dense_rank|
+----------+------------+-------+-----+----------+
|S012      |Swathi      |AI     |96   |1         |
|S014      |Divya       |AI     |94   |2         |
|S011      |Vijay       |AI     |90   |3         |
|S013      |Nikhil      |AI     |90   |3         |
|S015      |Manoj       |AI     |85   |4         |
|S003      |Priya       |BigData|95   |1         |
|S001      |Teja        |BigData|92   |2         |
|S002      |Rahul       |BigData|88   |3         |
|S004      |Anil        |BigData|88   |3         |
|S005      |Kiran       |BigData|82   |4         |
|S007      |Arjun       |Cloud  |95   |1         |
|S009      |Ravi        |Cloud  |95   |1         |
|S006      |Sneha       |Cloud  |91   |2         |
|S008      |Meena       |Cloud  |89   |3         |
|S010      |Pooja       |Cloud  |84   |4         |
+----------+------------+-------+-----+----------+


========== TOP 3 STUDENTS PER COURSE ==========
+----------+------------+-------+-----+------------+
|student_id|student_name|course |score|student_rank|
+----------+------------+-------+-----+------------+
|S012      |Swathi      |AI     |96   |1           |
|S014      |Divya       |AI     |94   |2           |
|S011      |Vijay       |AI     |90   |3           |
|S003      |Priya       |BigData|95   |1           |
|S001      |Teja        |BigData|92   |2           |
|S002      |Rahul       |BigData|88   |3           |
|S007      |Arjun       |Cloud  |95   |1           |
|S009      |Ravi        |Cloud  |95   |2           |
|S006      |Sneha       |Cloud  |91   |3           |
+----------+------------+-------+-----+------------+
```

```
========== LAG - PREVIOUS SCORE ==========
+----------+------------+-------+-----+--------------+
|student_id|student_name|course |score|previous_score|
+----------+------------+-------+-----+--------------+
|S012      |Swathi      |AI     |96   |NULL          |
|S014      |Divya       |AI     |94   |96            |
|S011      |Vijay       |AI     |90   |94            |
|S013      |Nikhil      |AI     |90   |90            |
|S015      |Manoj       |AI     |85   |90            |
|S003      |Priya       |BigData|95   |NULL          |
|S001      |Teja        |BigData|92   |95            |
|S002      |Rahul       |BigData|88   |92            |
|S004      |Anil        |BigData|88   |88            |
|S005      |Kiran       |BigData|82   |88            |
|S007      |Arjun       |Cloud  |95   |NULL          |
|S009      |Ravi        |Cloud  |95   |95            |
|S006      |Sneha       |Cloud  |91   |95            |
|S008      |Meena       |Cloud  |89   |91            |
|S010      |Pooja       |Cloud  |84   |89            |
+----------+------------+-------+-----+--------------+
```

========== LEAD - NEXT SCORE ==========

student_id	student_name	course	score	next_score
S012	Swathi	AI	96	94
S014	Divya	AI	94	90
S011	Vijay	AI	90	90
S013	Nikhil	AI	90	85
S015	Manoj	AI	85	NULL
S003	Priya	BigData	95	92
S001	Teja	BigData	92	88
S002	Rahul	BigData	88	88
S004	Anil	BigData	88	82
S005	Kiran	BigData	82	NULL
S007	Arjun	Cloud	95	95
S009	Ravi	Cloud	95	91
S006	Sneha	Cloud	91	89
S008	Meena	Cloud	89	84
S010	Pooja	Cloud	84	NULL

========== PREVIOUS AND NEXT SCORE ==========

student_id	student_name	course	score	previous_score	next_score
S012	Swathi	AI	96	NULL	94
S014	Divya	AI	94	96	90
S011	Vijay	AI	90	94	90
S013	Nikhil	AI	90	90	85
S015	Manoj	AI	85	90	NULL
S003	Priya	BigData	95	NULL	92
S001	Teja	BigData	92	95	88
S002	Rahul	BigData	88	92	88
S004	Anil	BigData	88	88	82
S005	Kiran	BigData	82	88	NULL
S007	Arjun	Cloud	95	NULL	95
S009	Ravi	Cloud	95	95	91
S006	Sneha	Cloud	91	95	89
S008	Meena	Cloud	89	91	84
S010	Pooja	Cloud	84	89	NULL

```
========== CUSTOMER POLICY DATA ==========
+-----------+-------------+--------------+-----------+
|customer_id|customer_name|policy_name   |policy_date|
+-----------+-------------+--------------+-----------+
|C001       |Teja         |Health Basic  |2025-01-10 |
|C001       |Teja         |Health Premium|2025-06-15 |
|C001       |Teja         |Health Gold   |2026-02-20 |
|C002       |Rahul        |Life Basic    |2025-03-12 |
|C002       |Rahul        |Life Premium  |2026-01-25 |
|C003       |Priya        |Auto Basic    |2025-02-18 |
|C003       |Priya        |Auto Premium  |2025-09-10 |
|C003       |Priya        |Auto Gold     |2026-03-05 |
|C004       |Anil         |Home Basic    |2025-05-20 |
|C004       |Anil         |Home Premium  |2026-04-01 |
|C005       |Kiran        |Travel Basic  |2026-02-15 |
+-----------+-------------+--------------+-----------+


========== LATEST POLICY PER CUSTOMER ==========
+-----------+-------------+-----------+-----------+-----------+
|customer_id|customer_name|policy_name |policy_date|policy_rank|
+-----------+-------------+-----------+-----------+-----------+
|C001       |Teja         |Health Gold |2026-02-20 |1          |
|C002       |Rahul        |Life Premium|2026-01-25 |1          |
|C003       |Priya        |Auto Gold   |2026-03-05 |1          |
|C004       |Anil         |Home Premium|2026-04-01 |1          |
|C005       |Kiran        |Travel Basic|2026-02-15 |1          |
+-----------+-------------+-----------+-----------+-----------+
```

```
========== CUSTOMER POLICY HISTORY ==========
+-----------+-------------+--------------+-----------+---------------+--------------+
|customer_id|customer_name|policy_name   |policy_date|previous_policy|next_policy   |
+-----------+-------------+--------------+-----------+---------------+--------------+
|C001       |Teja         |Health Basic  |2025-01-10 |NULL           |Health Premium|
|C001       |Teja         |Health Premium|2025-06-15 |Health Basic   |Health Gold   |
|C001       |Teja         |Health Gold   |2026-02-20 |Health Premium |NULL          |
|C002       |Rahul        |Life Basic    |2025-03-12 |NULL           |Life Premium  |
|C002       |Rahul        |Life Premium  |2026-01-25 |Life Basic     |NULL          |
|C003       |Priya        |Auto Basic    |2025-02-18 |NULL           |Auto Premium  |
|C003       |Priya        |Auto Premium  |2025-09-10 |Auto Basic     |Auto Gold     |
|C003       |Priya        |Auto Gold     |2026-03-05 |Auto Premium   |NULL          |
|C004       |Anil         |Home Basic    |2025-05-20 |NULL           |Home Premium  |
|C004       |Anil         |Home Premium  |2026-04-01 |Home Basic     |NULL          |
|C005       |Kiran        |Travel Basic  |2026-02-15 |NULL           |NULL          |
+-----------+-------------+--------------+-----------+---------------+--------------+


============================================
DAY 17 COMPLETED SUCCESSFULLY
============================================
```